Assignment 2: Engine Modules and Object Management

Description

Following the first assignment, we have successfully built an imprint of a skeletal structure for a game engine, now we build the bones – following through on our initial outline. This will involve building a few different *modules* for our engine. A module in this sense is a subsystem of the engine dedicated to a specific task. These modules are what will allow us to automate the processing of game objects.

Because these are disparate modules, there is not necessarily a clear indication of order to organize your workflow. The recommendation would be to build the baseline materials suggested here, and then if any *coupling* is desired, to add onto the modules/tools after the baseline is functioning. Baseline functionality for the various modules is as follows:

- Time step is updated
- Game objects
    - Memory is managed
    - Objects are updated per frame
- Rendering Graphics
    - Generalized mesh loading procedure
        - API functionality
    - Buffer management
        - VBO/VAO
        - Textures
        - Shaders
    - OpenGL state settings appropriately
- Rendering Audio
    - Managing an audio device
    - Event-driven object-based playback
        - Streaming active sounds
- Scene Graph Management
    - Data structure for fast queries with game objects

A few distinctions should be made to consider where these modules are disjoint to avoid unnecessary *coupling* of our subsystems. The *Game Object*s will need the current time step to update themselves – as most objects in a game engine are driven by a time differential. The *Scene Graph* is likely a good feature to complete at the end for two reasons: our initial tests of our game engines will not require optimizations like a good data structure for spatial data; secondly, the hierarchical transformations provided by the scene graph are not required to begin initial testing/game building with the engine.

Once these features are implemented, you will build a small game demonstration.

Specification(s)

Objective A: Time Updates and Program Organization

We have setup a logic loop for ourselves in which we will implement a *Discrete-event Simulation* model. Currently our loops poll SDL events, update our open windows and check other I/O states, and render a mesh in an OpenGL context – refreshing the window on a per frame basis. The next two steps to implement are *time increments* and *object updates*. Object updates are described in **Objective B**, and because updating a time step is straightforward process, we will also use this section to organize/refactor our programming space(s).

An Application Programming Interface:

As you develop the tools for your game engine, you will inevitably need to provide access to some of these tools for an *end user* – i.e., someone attempting to use your engine for game development. This also includes limiting access to other portions of your code. In a proper scenario, we would be writing our tools, compiling them, and using dynamic linking to avoid inadvertently giving access to protected portions of our software. However, as a general exercise, we can ignore this for now (which is always a valid option if our code is open source anyway).

Regardless of providing access to our code or not, *we should employ* some software engineering practices to ensure good *encapsulation* of data and programming space. Consider for instance our writing of an SDL Manager in Assignment 1 – we encapsulate window creation, updates, and destruction within the manager class and do not expose our window pointers to the external world. That is, we use the public methods of a class as an interface for what should be allowed, and therefore disallowed, within a section of code.

We can carry this idea forward by designating regions of your code for internal only tools and memory, as well as public interfaces (as expected in an API). The API functions might in fact be composed of some of the private tools and access private memory, but that is perfectly valid as an application layer for the end user need only see a header file of declared functions. This header file is where your API functions/features should be defined.

Take a moment to build this organization in your code, then we can go ahead and add the *time step* update for our DES system:

- *Utility* header and source file
  - For internal global variables and functions (hidden from the end user space)
- *Engine* header and source file
  - For external global functions, this is the API

        - Intention is for the source file to remain hidden
    - Including additional libraries for the user is useful here
        - E.G., GLM
- Source file for the end user's application layer
    - Define an *initialize()* function here
        - Declare this function inside the Engine API header

NOTE, the initialize() function needs to be visible to the engine's application layer (specifically, where the entry point *main* exists) so that the engine automatically invokes this function on boot. For term project considerations, *what should your default initialize function look like?* Consider that Godot uses its own engine to build a GUI interface to develop a game, which will also be executed by an instance of the engine. **While contemplating this, there is a requirement to meet for this assignment described in Objective F – involving the building of a small game.**

With the separation of the codebase, we can go ahead and compute a delta time for use with object updates. In your *main* function's logic loop, you will need to perform the following:

1. Define a *previous* and *current* time point outside of the logic loop.
2. At the start of the logic loop
    a. Update the *current time*
    b. Calculate a delta time
    c. Update the *previous time*
3. Store this integer (units of milliseconds) representation of delta time in a global utility variable
4. Store a float version of this delta time (units of seconds) in a global utility variable
5. Ensure either the integer or float time variables are accessible
    a. Each with their own API functions

The choice of unit for the integer time is expected to be milliseconds. Eventually you might do performance tuning based on microsecond analysis, but the general scope of games is in milliseconds. The global variables should not be accessible from the API header – instead being stored in the *Utility* files. The API source files that implement a *getDeltaTime* and *getDeltaSeconds* should be able to reference these variables by including the Utility header file. These functions signatures should be as follows:

- integer getDeltaTime() -> returns the integer delta time
- float getDeltaSeconds() -> returns the float representation of the delta time in seconds

The integer return type should appropriately match whatever type is observed when computing the initial delta time value: e.g., size_t, std::uint64_t, std::uint32_t, or even SDL aliases like Uint32. Note, the fixed integer types for the C++ ISO are in <cstdint>.

Until you get your game engine loaded with sufficient work, you will likely be dealing with increments in time faster than a millisecond. For this reason, go ahead and include the <thread> header in your engine's application layer so we can stall the *main thread* by 1 millisecond. The <chrono> library includes literals to conveniently represent statically typed numbers (i.e., a literal) as durations of nanoseconds, microseconds, nanoseconds, etc.. by typing a suffix alongside the literal value. To use these literal suffixes with ease, add a *using namespace std::chrono_literals;* at the top of your engine's application layer file. *Alternatively,* refer to the SDL_GetTicks() function.

The end of your *main's* logic loop should include a *std::this_thread::sleep_for(1ms);* to stall the main thread for 1 millisecond.

Objective B: An Abstract Class for Game Objects

Assuming you have built the separation of the codebase described in **Objective A**, then you will have the Utility header and source sections available. This objective requires two predominant tasks:

- Defining a class with polymorphic potential
- Storing an array of active GameObjects to be managed by the engine

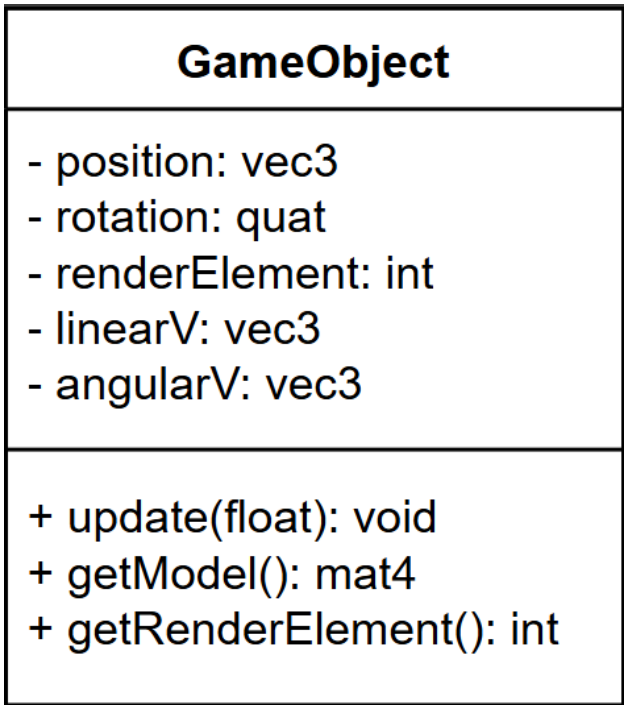
These two things are slightly coupled together depending on how you choose to implement polymorphism – **you will be required to try out at least two methods and to perform some benchmarking to justify choosing one path.** We will discuss some of these options in a moment, but first we must consider our goal: we want a list of game objects which the engine can automatically update in a for-loop, and for which the graphics rendering system can determine what to make draw calls for.

Given one of the state variables that will be required for the Game Object, an additional class is described as being relevant to implement – for which an additional set of performance tests should be performed.

Required Classes:

The first class that is required is the GameObject class. It needs to hold a minimum amount of information – under the premise that it exists within the global coordinate system of a game world, and has a render-able representation (i.e., is associated with a mesh). There are additional object types that a game engine will likely store – things focused on smaller coordinate spaces (i.e., 2D game objects), cameras as objects, bounding volumes, etc.. These are things you might want to implement in your game engine, but for the purposes of language, a GameObject is a 3D instance of an element visible within the game world. After building this utility and managing a buffer of these objects, feel free to incorporate more object types for an end user to utilize.

The GameObject class should have (at a minimum) the following:



The position and rotation field members represent state within the coordinate system. The rotation field is a *quaternion*, which is the second required class to complete in this section (so, more on this shortly). The *renderElement* is an integer that will be used to associate this GameObject with an element that can be rendered using OpenGL. If you prefer to use something other than an integer, that is fine; but note the general rendering procedure will be to iterate through the list of game objects to populate a render queue.

The *linear* and *angular* velocities will be used more in the physics section of the course, but you can use the linear and angular velocity for the time being to update the position and rotations of game objects for the demonstration game.

The *update* function will take in a float (the floating point representation of delta time in seconds); but it is acceptable to take in nothing if you want to enforce that the update function needs to call the API function from **Objective A**, getDeltaSeconds(). This function will be the interface to derived classes' implementation of user-defined game logic.

The *getModel* method will take the rotation and position field data to generate and return a 4x4 matrix to stream to a shader uniform before invoking a draw call. The getRenderElement() is a safety interface to the renderElement value – preserving encapsulation by removing unnecessary external write privileges.

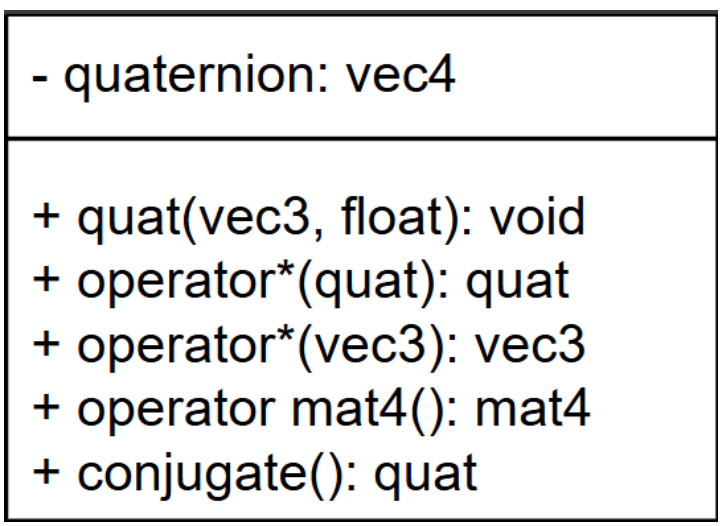
These features ensure a game object can have a time differential state, momentary frame state, arbitrary definition for being updated, and can be rendered as an object with a data type (mat4) that is most practical for the GPU.

Note, whatever update function is invoked from a GameObject, it is a method of the GameObject class which means it can access private data. This is important in considering encapsulation and data protection from external modification. Specifically, you can write functions in your game engine's API like:

void integrateVelocity(const glm::vec3& velocity, glm::vec3& position)

in which an argument might be modified. Assume that within the *update* function, this type of function is invoked passing in private member data that will be modified. This is generally data safe, and good encapsulation practice from a software engineering perspective. Specifically, no one is *taking* permission to modify what should be private data, an internal method that has permission to alter private data had to choose to invoke a method which was given access to alter this data. This practice will come up again in **Objective D** for rendering audio, but more on that in its section.

The next class is for having quaternion representation for orientation of objects, vectors/points, etc.. There is an additional quaternion class in the *GTC* section of *GLM*. You are welcome to use this, but for full points in the assignment, you will need to create your own quaternion class. If you do implement your own quaternion class, see the Wikipedia link to find the Hamilton Product; these are the most straightforward rules to implement the multiplication operator for a quaternion, but more on that after the UML:



The constructor for the quaternion should be information to generate a rotation from an axis and an angle. The axis must be normalized. The quaternion setup is simply, one component represents the "real" value, which is the ***cosine of half the angle*** and the three remaining "imaginary" components are the axis vector scaled by the ***sine of half the angle***. The half angle is what is stored because transforming a 3D position or vector by a quaternion happens with two Hamilton Products: q * p * q.conjugate() *where q is the quaternion to rotate a position vector p*. The two multiplications, where the first is the original quaternion and the second is the conjugate (i.e., the inverse rotation), effectively

performs half of the rotation into a higher dimensional space, while the conjugate multiplication will finish the rotation and eliminate the imaginary parts. After constructing the axis and angle portion of the quaternion, you will need to normalize it to ensure it is unit-length (we perform rotations with unit-length quaternions; effectively describing orientations about a unit hypersphere).

Note how this actually works as two Hamilton Products, the position we are transforming needs to be represented as a quaternion as well, **but with its "real" component set to 0.0f.** There is an alternative approach that is based on vector operations: the Euler-Rodrigues Formula. *Consider keeping your code "**dry**" by implementing the quaternion multiplication for the Hamilton Product and then reusing it for the multiplication operator with a vec3*.

The conjugate is the negation of the axis portion of the quaternion – i.e., typically referred to as the x, y, and z components (akin to the 3D Cartesian system). Note, that this conjugate will define a quaternion that can perform an inverse rotation – **i.e., it is guaranteed that you can invert a unit-length quaternion.**

These C++ operators are operator overloads – effectively, syntactic sugar by invoking a member function that is accessed symbolically. The *operator mat4()* is unique in that it is a *conversion function* to cast an object to another type. You can achieve the same thing with a conventional function that will return a mat4. To convert a quaternion to a mat4, see Martin Baker's solution (although, note the second solution supplied to Martin 😊).

GameObject Management:

You will need to come up with a proposal for establishing your GameObject class type. The most straightforward option is to rely on runtime type evaluations and function pointer lookups with the *virtual* operation. When a derived class inherits from a parent, the child can ensure polymorphic behavior by overriding a function signature that had been declared as virtual by the parent class.

With this, if you dynamically instantiate a derived class in a GameObject pointer, the virtual function overridden will be looked up at runtime to invoke the derived class method. This virtual function would specifically be for the *update* function, so that GameObjects can be specialized as different types. This incurs a runtime cost and some additional memory overhead as a virtual lookup table is created and used to find the appropriate function to call. There is a potential alternative to runtime polymorphism evaluation – defining all possible derived classes available to an end user. Note why this can still be a dynamic/viable option:

- Objects can be downcast to an appropriate derived type
    - Switch statement
    - Curiously recurring-template pattern

- Derived classes can still have custom (scripted) behavior

The former statement describes a static conversion process. The latter statement is because functions can be passed as pointers. That is, a derived class could invoke a function pointer inside an update() function call – likely passing in the derived class/field data to modify stored states.

Note that we cannot only rely on function pointers for the GameObject class – assuming that future GameObjects need additional memory for specific state storage and object behavior (e.g., humanoid with N limbs). However, if all GameObjects are going to use the exact same amount of memory, then we can employ use of function pointers alone in the GameObject class.

Note: YOU WILL BE REQUIRED TO COME UP WITH A TENTATIVE PLAN FOR POLYMORPHIC BEHAVIOR WITH GAME OBJECTS. You will discuss this during your Assignment 1 evaluation meeting – at which point I can suggest the second method to implement for your benchmark tests.

See slides from Lecture 9 to see where your DES system should update GameObjects. The actual container storing, for instance, GameObject* should be considered to be a basic array (for performance), but could very well be a std::vector container. If you use a basic array, you will need to define a MAX_OBJECT count (size_t) and will need to keep track of how many objects currently exist. With the static array, consider use of a *swap-and-pop* procedure. Additionally, if dynamically creating GameObjects, be sure to to call **new** and **delete** appropriately.

Benchmark Tests:

The benchmark tests will have you evaluate performance differences between at least two approaches to polymorphic behavior (please limit this to two until the majority of the assignment is complete). This will require a small technical report to be written including a chart describing the difference. The report is as follows:

- Methodology
  - Independent and Dependent Variables
- Metric
  - Units of time
- Graph(s)
  - Visual comparison
- Conclusion
  - What method will you utilize?

The report should be around 1-page. You methodology should be sound – for instance, both methods using 50 objects, each with the same update functions. You will need to run

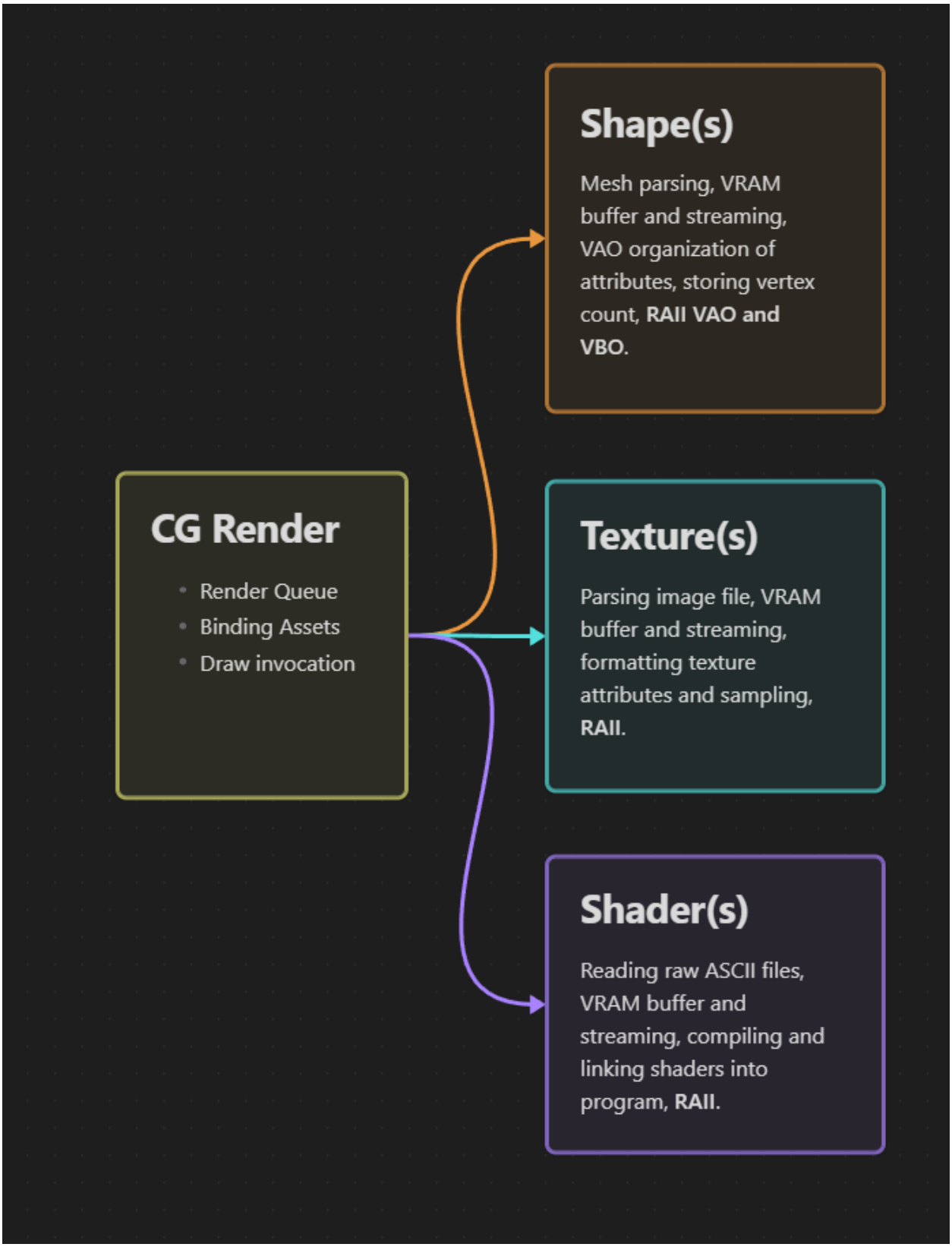
the polymorphic behavior several thousand times to assess general performance differences: for instances, run the logic loop 10,000 times. You should collect the delta times per iteration into an array which can be written as a CSV to a file and processed in a Spreadsheet application.

Objective C: Rendering Objects

You have already written most of the code necessary to load arbitrary meshes from your own filetype, but you will need to add more information to this and ensure that your Shape class (or equivalent) also manages vertex attributes appropriately. *There are a few options available for you, with one option offering an extra point*.

At a minimum, you can define exactly one type of mesh structure for your game engine. I.E., all meshes will have the same number/structure of vertex attributes. For an extra point, you must extend your mesh filetype to include additional header information to discern how many vertex attributes exist for a mesh, and how many pieces of data are involved (so, for example, you could read one mesh with position and normals, and another mesh that has positions, normals, and uv coordinates). To receive the extra point, the rest of the rendering system must also be fully functional.

Rendering the game world should be defined in a class, *why*, because there are object states to manage and protect from external users. Effectively, the code should be more maintainable and reliable to write in a class. Furthermore, this means you could have different Render objects that are devoted to rendering specific types of objects – e.g., 3D game world, transparent FX, 2D GUI, etc.. The actual class implementation is up to you, but will be evaluated based on its ability to function and the soundness of its initial design for software – the latter being defended by your rationale during the Assignment 2 review/interview. Here is the general outline to consider when planning a design for this module:


Shape(s)
Mesh parsing, VRAM buffer and streaming, VAO organization of attributes, storing vertex count, RAII VAO and VBO.
CG Render
Render Queue
Binding Assets
Draw invocation
Texture(s)
Parsing image file, VRAM buffer and streaming, formatting texture attributes and sampling, RAII.
Shader(s)
Reading raw ASCII files, VRAM buffer and streaming, compiling and linking shaders into program, RAII.

This exercise is effectively a refactoring of your previous mesh/shader prepping and rendering procedures. There is a new type of data and OpenGL buffer to manager – a Texture buffer, used for sampling image data inside a shader.

Texture Buffers:

The texture is managed exactly as you had seen with other OpenGL data:

1. Request a buffer
    a. glGenTextures(1, &textureName);
2. Bind the buffer
    a. glBindTexture(GL_TEXTURE_2D, textureName);
3. Stream data to the buffer
4. Set texture sampling parameters
    a. glTexParameteri should be sufficient for this..
    b. At a minimum, consider setting texture wrapping behavior
5. Unbind the texture until ready for use

The texture parameters assert what behavior to use in certain situations. A texture is normalized to coordinates between [0, 1] so that the actual resolution of the texture is agnostic to the sampling of values in the texture. This does mean that textures can be sampled in between existing texels, or possibly outside of the range of [0, 1]. So, these parameters describe what should occur – for instances, clamping a value to a texture edge, repeating or mirroring a texture when outside of valid coordinates, and also how to apply mipmapping. Mipmapping can be included automatically in OpenGL: it will generate a sequence of lower resolution texture samples for you with glGenerateMipmap. Note that mipmapping helps reduce aliasing when viewing a texture at a distance – instead showing a lower resolution (blurry) image which will have less drastic texel sampling variance when in motion. It is up to you if you want to manage mipmapping in your engine – which can, notably, be performed manually by loading each mipmap layer.

Using a texture is a bit simpler than utilizing a new *uniform* variable – however, a texture to sample in a shader is defined as a *uniform* type – not because we will necessarily stream texels per shader invocation but because we will establish for the shader which texture is currently bound for an invocation. **NOTE,** this is nice because it means we can arbitrarily change the texture that is bound and sampled by a shader program. *Refer to lecture slides for more information on texture binding*, but as a reminder – as we are default using OpenGL 4.1, we do not have guaranteed access to the verbose description of binding indices. Without these layouts, we technically need to set the texture unit binding index by fetching the uniform from a shader. For a simple floating point 2D texture, you can use a *sampler2D* in GLSL.

As you performed before, we can glGetUniformLocation from a shader program to look up a texture uniform. Once looked up, we can use:

glUniform1i(textureUniform, 0) to stream 1 integer to the shader – effectively defining this texture as *texture unit 0*. You would then do these for up to N texture units for a shader, incrementing the integer value streamed to the uniform.

Once textures have a texture unit ID, readying a texture before a render call is as simple as activating a texture unit ID and binding a texture:

- glActiveTexture(GL_TEXTURE_0 + texturesUnitID)
- glBindTexture(GL_TEXTURE_2D, textureName)

Note the state machine behavior of OpenGL. Setting an active texture unit ID distinguishes which unit will be modified when we make a bind call. So, you will need to group these API calls together accordingly when rendering an object with textures.

CONSIDER POTENTIAL BUGS! This is a good time to start considering if there will ever be any detriment to utilizing OpenGL for rendering, then initializing new assets before rendering again. I.E., when and where is it appropriate to reset state machine values and are there unnecessary places to do so – thereby minimizing latency in frametimes?

Streaming the texture data will require loading a texture file. For a basic bitmap, you can use a built in function from SDL: SDL_LoadBMP. This function takes in a C-string for the filepath to a bitmap and returns a SDL_Surface*. This memory was dynamically allocated, so after streaming the surfaces pixels to OpenGL buffer, you will need to SDL_FreeSurface to deallocate this memory.

If you want to support additional image types, then consider utilizing the SDL_image features. This will involve linking in more external code – building the dependencies of your engine, but it ensures a similar structure where images can be loaded into SDL_Surface objects and streamed to the GPU. Note why bitmaps are supported by default – they are the simplest files to read as no decompression is required. When we upload information to the GPU, it will be in its full (raw) form. Meaning if we utilize PNGs, we get the memory footprint reduction benefits when distributing a game, but when we read the file and stream it to the GPU, it will be occupying its full memory footprint. Note that there are some built in texture compression features in rendering APIs – but only look further into these based on time available.

Class Structure:

When designing your CG rendering system, consider a few things: you do not necessarily know how many objects will be rendered per frame (albeit it is deterministic in a time step during runtime), and excessive state changes will influence framerate. The latter is a detail that can be resolved with refactoring for optimizations, but is still useful enough to consider in our initial implementations in the hope of easily modifying our codebase to increase performance later.

For the time being, consider the most extreme option an end user might desire – given meshes with equivalent layouts of vertex attributes, they desire the ability to arbitrarily permute between all shader programs and textures available for each mesh during the lifespan of an application.

Do not build for this situation immediately, but consider why I bring it up. When building these engine tools, we are deploying interfaces that define what is possible for a game developer. This means, we have to think about what that client/end user might want as a feature. There is a big caveat here to address as well – *legacy bias*. You may have been exposed to existing video game products and used game engine tools, BUT these experience do not necessarily guarantee what is a correct choice/solution for building a game. This is to say – mimicking the structure of other tools entirely does not provide any new perspective in game design and creation. *Having you design this otherwise complex class structure gives you an opportunity to determine how to solve a problem – to allow flexibility for a game developer – while also leaving divergent paths open for something unique and special to be attempted*.

Some ideas to venture in your graphics rendering engine design:

- Singleton with interface to push VAOs into a global render queue
- Separate renderer per graphical element type
- Separate renderer per shader program
- Render class is-a vs has-a
    - Manage multiple shader programs, meshes, textures, etc..
    - Utilize *multiple inheritance* is a Renderer is a program, mesh, texture, etc..

Try to design a simple choice initially and move forward with that. Further modifications and expansion can happen after the fact – but keeping to *clean coding* and *software engineering* practices of minimizing coupling and maximizing cohesion will help make this codebase extendable, reusable, and maintainable.

MVP Transformation:

One final requirement is to ensure that what we have built previously is more capable for general gameplay features. Notably, we defined a perspective (frustum) matrix to ensure that we would have a monocular depth cue in our OpenGL rendering. However, as discussed in the first Quarter of class, we typically employ the use of three matrices to render computer graphics: a model transformation (for local to global space), a camera (view) transformation, and also a projection transformation (e.g., the perspective matrix).

So, your general scene shaders will need to incorporate these 3 transformations to generate an MVP transformation for vertices. Your choice of uniforms and uniform buffer objects within your render system is up to you.

Objective D: Rendering Audio

Rendering audio is a similar practice to rendering computer graphics. You will need a queue of sounds to play – which can be added to over time – but the major difference is the temporal component to rendering sound.

Computer graphics are rendered on a per frame basis. Effectively, we can generate a render queue, invoke draw calls for everything in the queue, and start the next frame with an empty queue. The temporal coherency of the visuals on screen are based on minor spatial and color updates to objects over the elapsed frames. Audio from a file is instead a strict set of discretized amplitudes which appear as a wave over time. If we were to procedurally generate sound, then the process would be extremely similar to rendering graphics: we would store objects to update, alter there current frame state amplitude, and push this to a render queue just for the frame. However, we do not always want to procedurally generate sound waves – we want to playback prerecorded music and foley FX in an object-based manner. Even if we did procedurally generate sound, we are still at the mercy of how audio is played back by a sound card or audio interface – they convert discretized amplitudes into continuous waves, doing so by having a buffer of data populated and played back over time. This audio buffer size means we do not truly get to do one-off render operations per frame as we did with the graphics pipeline.

SDL has a separate SDL_mixer library to assist in interfacing with the sound card, but this is a senior-level project based course. We are going to dig into the weeds a bit with using different hardware devices in an application. For this reason, we will use barebones features provided by SDL to open and audio device, declare a callback function to stream audio, chunk the audio, and manage playback of up-to N-audio objects at a time.

To help you get started in this process, some initial guidance is provided for the class structure – but the actual implementation will still be up to you.

SoundSystem

- sounds: vector<Sound>
- device: SDL_AudioDevice
- playback: vector<SoundStates>
- mix: Uint8*

+ SoundSystem(): void
+ ~SoundSystem(): void
+ loadSound(string): bool
+ playSound(int): bool
+ playSound(string): bool
- callback(void*, Uint8*, int): void

Breaking down this system top-down:

- sounds
  - This is a library of preloaded sounds ready for playback
  - Akin to our preloading of meshes in GPU VRAM
  - Typically small sound objects for event-driven object-based rendering
- device
  - This is the audio device opened by SDL
  - SDL_OpenAudioDevice
    - RAII required for creation and destruction
  - This device will remain active for the lifespan of the audio system
- playback
  - These are lightweight objects storing the state of actively playing sounds
    - This a good use for a struct
    - Need only store a Uint8* and an integer for length
      - This is storing the index position of a Sound object

            - The length integer is originally set to the size of a Sound object
                - Will increment the index pointer and decrement the length integer
                - Determine when a sound ends
- mix
    - This is an array that will be sized according to the audio buffer
        - I.E., you will dynamically allocate this buffer space when you know the size of the audio buffer stream provided by the audio device
    - In a callback, you will initialize these values to 0
        - Then iterate over *playback* SoundStates to mix in additional audio chunks
- SoundSystem
    - Constructor which will open the audio device
        - Potentially allocate the size of the *mix* array
    - Assuming no errors, unpause the audio device opened
        - SDL_PauseAudioDevice
            - Set second argument to 0 to unpause
            - This will remain unpaused for the lifespan of the application
- ~SoundSystem
    - Destructor will pause the audio device
    - Cleanup any loaded audio
    - Cleanup *mix*
    - Close the audio device
        - SDL_CloseAudioDevice
- loadSound
    - Argument is a filepath to a WAV file
    - Will load the WAV data and create a Sound object that is pushed on the *sounds* vector
    - Returns true if successful, else false
- playSound
    - Int argument will play a sound where the integer should reference an index in the *sounds* vector
        - Note, you can change this mapping and container if you so choose
    - String argument overload will load a sound not in the *sounds* container and play it back
        - This will require additional planning on your part
        - Expected behavior is to only support one runtime loaded sound at a time
            - This is useful for a large track
                - Dialogue

        - Music
        - Etc..
- callback
    - This will need to be a free function (by some means)
        - It will be passed to your SDL_AudioSpec callback pointer

The *Sounds* and *SoundStates* types are up to you to design. But, the primary recommendation is for SoundStates to be a struct with minimal information for an active sound being played; and for Sounds to be a class that will utilize RAII principles. The RAII features for the Sound class is important given the use of loading and freeing dynamically allocated arrays for WAV files.

WAV files can be loaded with SDL_LoadWAV and freed with SDL_FreeWAV. The loading function is very C-like in that it takes in a memory addresses for its last three arguments – modifying the third and fourth arguments on successful loads. A WAV file is an uncompressed set of samples of amplitudes – sampled at some frequency. The minimum amount of information required to actually output sound is just the array of sound samples, and a length variable (bytes) to know how many samples are in the array.

The callback function takes in a void* for its first argument – which will require you to determine what type of object your want to pass to this function. The function will need to access the *mix* buffer (or generate a new one every time callback is invoked) as well as the list of actively queued sounds in *playback*. The second argument is the buffer that the sound card will be playing information from, and the third argument is the number of bytes that this buffer can take in.

When starting the callback function, we want to generate a mix array of all zeroes to ensure that if no sound is playing, then the audio buffer will not produce any output. From there, you will iterate over the active SoundStates in the *playback* container to add in N-bytes of data – where N is equal to the length of bytes for the audio buffer. If the sound has fewer bytes than the length of the audio buffer, then you can break from an iterating loop or just add zeroes. Upon completion of this SoundState, you will need to dequeue it someway from your container. Mixing the SoundState to the audio buffer is simply a for-loop:

- For i in 0 to len:
    - mix[i] += SoundState.buf[i]
- SoundState.remaining -= len;

Of course, add the additional conditions needed to not go out of bounds in the for-loop and to ensure that if len is greater than remaining, to remove the SoundState from the playback queue.

The primary challenge here is to preserve an open sound stream that you will mix all samples into. This is an extremely low-level perspective of audio playback that lets you see

behind the scenes a bit more on how this type of system/hardware works. **You are welcome to modify the class structure a bit, *and will need to do so to support loading a sound for playback that is not in the list of sound objects***. At a minimum, your sound system must be working with monoaural WAV files (**no stereo WAV files!!!**) and must be mixing to at least a 2-channel sound system – meaning you will need to manage indexing appropriate when summing into a the *mix* array from the SoundState buffer – Left and Right channels are interleaved, see the audio spec information for more details; but this effectively requires adding one byte from the monoaural chunk to mix[i] and mix[i+1].

Objective E: Scene Graph class

We will need a scene graph for several reasons going forward: it can be used for render culling, setting up bones for a rigged armature, define accelerated collision detection for physics processing and more. What scene graph you want to build will be specific to your own desires for your game engine. With that said, we will be covering scene graph strategies – for which you will pick an option for your general structure. However, you will need to support at least one *tree-like* data structure in order to support animating meshes with bones.

The options for your general spatial graph structure are as follows: uniform grid, binary spatial partition, octree, k-d tree, or binary spatial partition.

We will expand upon this further for the physics section – so the predominant requirement is to build a basic structure to support accelerated render traversal (i.e., minimize the number of frustum culling tests required for rendering) and to have a tree ready to use for nesting spatial transformations.

Note, the scene graph will not be the same as your general GameObject memory pool (or the like), but instead is a representation of the elements in the memory pool organized in a spatial way. The predominant reason is in many scenarios *it will be faster to reconstruct a scene graph compared to reorganizing every single element managed by the game engine* AND *we still exploit potential caching mechanisms with spatially adjacent information in the memory pool*. ***Furthermore, you will eventually want to create GameObjects that have a temporary lifespan, BUT, these objects will still need to play nicely with the rest of the system(s)***.

Your basic Scene Graph structure should minimally be:

Node
- children: Node* - objReference: ???
+ Render(Render Queue): void

The *objReference* is however you want to manage referencing an object from your GameObject memory. The Render method is meant to perform frustum culling to determine is an object should enter into a RenderQueue. Depending on how you choose to implement your render system, these input argument will vary, and if you choose, you can make traversal through your scene graph and external operation in a private Utility function. The Node pointer can be generalized for both uniform grids, and lists of nested objects – because all of these things are just arrays. This abstract description is open for you to change, but indicates the classical linked-list potential of your data structure.

A parent-child inheritance of transformations is expected – such that as you iterate down branches of the nodes, you will carry forward the transformation of the parent to modify the children.

When you implement this in your engine, I will expect to see documentation of the structure *you* implemented during the review meeting.

Objective F: Build a Classic Arcade Game

Even without the scene graph as a fully featured component of your engine, you have all of the tools necessary to illustrate the DES model for updating time and states. The amount of states updated – GameObjects, visuals, and sound – is sufficient to build a simplistic videogame. So, the last step of this assignment is to utilize everything you have worked on here to build a video game.

This objective exists for a few reasons:

- Unit/integration testing of engine features
    - Integration testing via future implementation changes
- Verify and validate what must exist

The latter portion is critical. If you only focus on tools and inferred potential, then you can forget to add important features or overthink systems that do not need the complexity. Building a small game should ground your work, forcing you to keep things simple – verify

that this tools can be used to build a game – and to consider what functions should exist both in the private Utility section of code and the public facing API.

This is to say, as you work on this game, you will start to see functions that are useful for backend and frontend systems. This is the portion of the assignment where you should be able to fill-in-the-blanks and decide what you would like regarding game engine tools as a game developer. While we go over the general architecture of game engines, note again that every feature you have seen in another game engine is a specialization that fits into this architecture. Remaking every utility verbatim that you have seen before is not necessarily fruitful for the *uniqueness* of your engine. Of course, exploring and implementation of some engine tools is useful to understanding, but the point here, by example, is if an abstract goal is AI path planning, then maybe you do not need to implement A* if your engine defines a way to work exclusively with parametric paths (iterative converging algorithm vs constant time execution).

Some recommendations for a "classic arcade game" are things like: Pac-Man, Dig Dug, Snake, Space Invaders, Asteroids, Frogger, but you can review and exhaustive list on Wikipedia.

Points

The point breakdown is as follows:

- Objective A is 5 points
  - 2 points: Time is Incremented
  - 2 points: Codebase is effectively segmented
  - 1 points: Time API functions exist and work as expected
- Objective B is 10 points
  - 3 points: Base GameObject class(es) exists and has expected working features
  - 3 points: Polymorphic-like behavior exists to update specialized objects
  - 1 point: Quaternion class exists and works as expected
  - 2 points: Technical performance report complete (alternative option available)
- Objective C is 10 points
  - 4 points: Render system defined that manages all VAO, Textures, and Shaders
  - 5 points: Can load and render arbitrary meshes, shaders, textures
  - 1 point: Utilize MVP transformation in your shaders for 3D objects
- Objective D is 10 points:
  - 5 points: Audio device opened, always plays until destructor called, and mixing happens correctly from monoaural audio output to at least stereo channels

    - 3 points: Management of pre-loaded audio is functioning as expected
    - 2 point: At least one dynamically loaded audio file is functioning as expected
- Objective E is 5 points:
    - 5 points: A scene graph class is defined that can be used to perform render culling and allow for parent-child transformation inheritance
- Objective F is 10 points:
    - 10 points: You replicated a historical game example, and were able to do so utilizing tools you built within your engine (API invocations from end user's application layer)